KERNEL BORDERLANDS

Bug Report

A Source-Verified Audit of Open, Resolved, and Historical Defects

Compiled for August 2nd, 2026

Scope

Direct source audit across all four subsystems — `kb-core`, `kb-aads`, `kb-control-plane`, `kb-op` — cross-checked against docs/development/core-control/control-plane-catalog.md and `CHANGELOG.md`. Every finding was independently verified by reading the current source directly, not taken on faith from prior documentation.

Kernel Borderlands Team

Karthik	PardhuVarma	Tejaswini	Rupa
2311CS040080	*2311CS040089*	*2311CS040077*	*2311CS040082*

Contents

Introduction

This report covers real, code-level defects and security gaps in the Kernel Borderlands product itself — sourced from docs/development/core-control/control-plane-catalog.md (the control plane's own severity-graded *Open Gaps* register) and `CHANGELOG.md`, cross-checked directly against the current source to confirm what is actually still open versus already fixed since those documents were written.

Dev-environment and tooling issues encountered during development are intentionally excluded from this report — see docs/reports/kb-aads/containment-rl-2026-08-02.md for those. This revision adds a direct source audit across all four subsystems (`kb-core`, `kb-aads`, `kb-control-plane`, `kb-op`); every finding below was independently verified by reading the actual current source, not taken on faith from any prior document.

1 Open — Confirmed Still Present in Current Source

BUG-001 — Unauthenticated, Wildcard-CORS HTTP API Can Terminate/Restore Containment on Any PID

Severity: Critical (Security)
Component: `kb-control-plane/internal/controlplane/{controlplane.go,http.go}`, `triggeredfromkb-op/kb-dashboard/src/App.tsx`
Status: Open

Description. `kbd` starts an HTTP API on `:8080` — bound to **all interfaces**, not loopback-only — serving `/api/isolate` and `/api/restore` with **zero authentication**, and a CORS handler that sets `Access-Control-Allow-Origin: *` on every response.

Confirmed in source:

```
// controlplane.go:184
if err := cp.StartHTTPServer(":8080"); err != nil {
    // all interfaces, not 127.0.0.1:8080

// http.go:46
w.Header().Set("Access-Control-Allow-Origin", "*")
    // every origin, no check

// http.go:398-423, handleIsolate -- no auth check anywhere
var req struct{ Pid uint32 `json:"pid"` }
json.NewDecoder(r.Body).Decode(&req)
s.cp.enforcer.Contain(req.Pid, uint32(pb.ContainmentLevel_TERMINATE),
    "Manual isolation from dashboard")
```

`handleRestore` (same file, near line 425) is identical in shape, calling Contain(..., ContainmentLevel_NONE, ...).

Impact. Any actor with network reach to port 8080 — or, because of the wildcard CORS header, *any webpage a browser loads at all*, from any origin — can POST `{"pid": N}` to `/api/isolate` and terminate containment on an arbitrary PID with no credential, token, or origin check whatsoever. This is strictly worse than BUG-011/BUG-012 (which at least require reaching a Unix socket): this is an open TCP listener with explicit cross-origin permission, reachable from anywhere the port is routable. Combined with BUG-011 (no protected-PID gate on the Go side), this path can also be pointed at PID 1 or `kbd`'s own PID with no additional resistance. Not mentioned anywhere in `control-plane-catalog.md`, which only discusses this daemon's gRPC-facing gaps — its HTTP API's auth posture appears to have gone unreviewed entirely.

Suggested fix. Bind to `127.0.0.1:8080` unless explicit remote-dashboard access is configured; require a real auth token/session on `/api/isolate` and `/api/restore` (and any other state-mutating route); replace the wildcard CORS header with an explicit allowed-origin list.

BUG-002 — UpsertProcessState Silently Resets an Actively-Contained Process Back to NONE

Severity: Critical
Component: `kb-control-plane/internal/store/process.go`
Status: Open

Description. `UpsertProcessState` — called on the hot path, continuously, for every incoming telemetry update from `kb-core` — builds a **brand-new** `CachedState{...}` literal and overwrites L1 wholesale. Unlike every other mutator in this file (`SetContainment`, `SetZone`, `InsertZoneTransition`, all of which correctly load-then-mutate-one- field), this one omits the `Containment` field entirely, so it is always the Go zero-value (`0 = NONE`) after any telemetry update.

Confirmed in source:

```
// SetContainment (correct pattern):
if v, ok := s.l1.Load(pid); ok {
    updated := *v.(*CachedState)
    updated.Containment = level
    s.l1.Store(pid, &updated)
}

// UpsertProcessState (the bug):
s.l1.Store(msg.PID, &CachedState{
    PID: msg.PID, PPID: msg.PPID, UID: msg.UID, Comm: msg.Comm,
    StartTimeNs: msg.StartTimeNs, LastUpdatedNs: msg.LastUpdatedNs,
    DimScore: msg.DimScore, CompositeScore: msg.CompositeScore,
    EMAScore: msg.EMAScore,
    SyscallEntropyLifetime: msg.SyscallEntropyLifetime,
    Zone: msg.Zone, EventCount: msg.EventCount,
    // Containment: not set -- zero value, i.e. NONE
})
```

Impact. An operator (or agent) calls `SetContainment(pid, TERMINATE)` → correctly recorded. `kb-core` keeps sending routine telemetry for that PID (containment does not stop monitoring) → the very next `ProcessState` update overwrites L1 with `Containment: NONE`. Every reader of `GetProcessState` — `kb-tui`, `kbctl`, the dashboard, `kb-aads` — now reports an actively-contained process as uncontained, within one telemetry cycle (typically sub-second). This is a state-consistency bug with direct operational-security impact: the system's own status reporting cannot be trusted for containment state.

Suggested fix. Load the existing `CachedState` and copy forward `Containment` (and ideally do a true partial-field update, not full replacement) the same way `SetContainment`/`SetZone` already do.

BUG-003 — JJE Consensus: Jury's Containment Vote Can Never Trigger on Any Real Input

Severity: High
Component: `kb-aads/consensus/jje.py,JuryAgent.evaluate_and_vote`
Status: Open

Description. `vote = "CONTAIN" if score > 75.0 else "ALLOW"`, where `score = alert_payload.get("confidence", 0.0)`. Every other confidence value in this system is a 0.0–1.0 float — confirmed against kb-control-plane/internal/controlplane/grpc.go's `SubmitAgentDecision` (`"confidence %.2f below 0.85 threshold"`) and `kb.proto`'s confidence fields. A real alert with confidence 0.95 (about as certain as this system ever gets) is not > 75.0 — so under the scale used everywhere else, this condition is never true for any realistic payload.

Impact. The Judge → Jury → Executor consensus containment path — the mechanism this project's own architecture treats as the primary quorum-gated containment decision — is effectively dead code. Jury never votes CONTAIN, so `coordinate_consensus`'s `contain_votes / total_votes > 0.5` check can never pass, so `execute_quarantine` is never triggered via this path. The identical threshold also appears in docs/development/control-aads/aads-development-plan.md:300, suggesting it was copied from an early design sketch and never reconciled with the 0–1 convention the rest of the system settled on.

Suggested fix. Change the threshold to the 0–1 scale (e.g. `score > 0.75`), and add a test asserting a realistic high-confidence payload (`confidence: 0.9+`) actually produces a CONTAIN vote — the kind of test that would have caught this immediately.

BUG-004 — kb_lsm_file_open Fails Open on Path-Resolution Failure, Before the Containment-Level Check Ever Runs

Severity: High
Component: `kb-core/ebpf/kbd_sensor.bpf.c`
Status: Open

Description.

```
int len = bpf_d_path(&file->f_path, path_buf, sizeof(path_buf));
if (len < 0) return 0; // allow -- before containment level is checked
...
if (level) {
    if (*level >= 3) {
        return -13; // Block ALL file open requests (full sandbox quarantine)
    }
    ...
```

If `bpf_d_path()` fails (a real, documented failure mode for certain special files, pseudo-filesystems, or under kernel memory pressure — not hypothetical), the function returns "allow" immediately, before the containment-level ≥ 3 "block all file opens" check ever executes.

Impact. A process placed at containment level 3 or 4 — which the code's own comment

states should have "ALL file open requests" blocked for full sandbox quarantine — has that guarantee silently bypassed for any file whose path resolution triggers this failure mode. This is distinct from the already-fixed zero-padding bug (HIST-3, which was about sensitive-path *string matching*); this is about the level-3+ block never running at all in this code path.

Suggested fix. On `bpf_d_path()` failure, fail closed for any contained process (check `level` first, before attempting path resolution, and block/return `-13` for level ≥ 3 regardless of whether the path was resolved) rather than allowing by default.

BUG-005 — VerifyChain() Discards the SQL Scan Error, Risking a False Tamper Verdict

Severity: Medium-High
Component: `kb-control-plane/internal/audit/audit.go`
Status: Open

Description.

```
rows.Scan(&ts, &action, &subject, &actor, &reason, &prevH, &entryH)
```

The returned error is never checked. On a scan failure (unexpected NULL, type mismatch, a corrupted row), the loop proceeds with zero-valued fields, hashes garbage, and reports `"chain broken at entry N"` — a false tamper alert — rather than the actual I/O/schema error. This directly undermines RESOLVED-1: the verifier that was just wired up has a latent bug that can produce an incorrect verdict.

Suggested fix. `if err := rows.Scan(...); err != nil { return false, count, err }`.

BUG-006 — kb-aads Jury Pool Size Is Hardcoded, Silently Ignoring Its Own Config File

Severity: Medium
Component: `kb-aads/consensus/jje.py,JudgeAgent.coordinate_consensus`
Status: Open

Description. `config/agents.yaml` declares `jury: pool_size: 5`, but `coordinate_consensus` hardcodes `jury_pool = [JuryAgent.remote(f"jury-{i}") for i in range(5)]` and never reads that config value. Currently invisible only because the numbers happen to match by coincidence.

Impact. An operator changing `pool_size` in config, expecting it to take effect, silently gets no change at all.

Suggested fix. Read `config/agents.yaml`'s `jury.pool_size` instead of the literal `5`.

BUG-007 — Consensus Round Has No Fault Tolerance for a Crashed Juror

Severity: Medium
Component: `kb-aads/consensus/jje.py,JudgeAgent.coordinate_consensus`
Status: Open

Description. `votes = ray.get(vote_futures)` has no error handling. If any spawned `JuryAgent` dies or its `.remote()` call raises, the exception propagates uncaught and kills the entire consensus round.

Impact. One crashed/misbehaving juror blocks the whole containment decision for that alert, rather than the round degrading gracefully (excluding the failed juror, re-normalizing remaining weights).

Suggested fix. Wrap in a per-future try/except (or `ray.wait` with a timeout) and proceed with whichever jurors actually responded.

BUG-008 — Unbounded Per-PID eBPF Maps Are Never Cleaned Up, Unlike Their CPM/CWP Siblings

Severity: Medium
Component: `kb-core/ebpf/kbd_sensor.bpf.c`
Status: Open

Description. `kb_handle_exit` explicitly deletes `protected_pids_map` and `protected_workloads_map` entries on process exit (correctly, for PID-reuse safety) — but three other PID-keyed maps are never touched by any exit path: `kb_syscall_totals` (10240 entries), `kb_syscall_counts` (655360 entries), `kb_cred_prev` (10240 entries). All `bpf_map_update_elem` calls against them also ignore their return value.

Impact. On a long-running host with normal PID churn, these maps fill permanently. Once full, new-PID inserts silently fail (`-E2BIG`, unchecked) — syscall-volume telemetry and the privilege-escalation UID-diffing in `kb_handle_commit_creds` stop updating for any new process from that point on, with no error surfaced anywhere.

Suggested fix. Delete corresponding entries from all three maps in `kb_handle_exit`, matching the pattern already used for `protected_pids_map`/`protected_workloads_map`.

BUG-009 — Dashboard Hardcodes http://localhost:8080, Compounding BUG-001 if Ever Pointed at a Real Host

Severity: Low (Medium once BUG-001 is fixed and this is deployed remotely)
Component: `kb-op/kb-dashboard/src/App.tsx(7callsites)`

Status: Open

Description. The dashboard's API base URL is hardcoded to `localhost:8080` rather than being environment-configurable.

Impact. Breaks the dashboard for any non-localhost deployment today. More importantly, it means the natural fix anyone reaches for ("just change the hardcoded URL to the real host") makes BUG-001's vulnerability remotely exploitable instead of LAN-local, unless BUG-001 is fixed first.

Suggested fix. Externalize to a build-time/runtime environment variable.

BUG-010 — AgentDecision.AuthorizedBy Is Captured but Never Validated

Severity: High (Security)
Component: `kb-control-plane/internal/controlplane/grpc.go,SubmitAgentDecision`
Status: Open

Description. `AuthorizedBy` is logged into the audit trail as if it were a meaningful authorization claim, but it is just a caller-supplied string field on the incoming `AgentDecision` message — nothing checks it against an allowlist, a signature, or any real authority before acting on it.

Confirmed in source (`grpc.go`):

```
cp.audit.Log(
    fmt.Sprintf("AGENT_%s", d.Action),
    fmt.Sprintf("pid=%d agent=%s conf=%.2f auth=%v",
        d.Pid, d.AgentId, d.Confidence, d.AuthorizedBy),
    d.AgentId, "",
)
```

`d.AuthorizedBy` is read into the log string and nowhere else.

Impact. Combined with BUG-011, any process able to reach `kba.sock` can submit a fabricated `TERMINATE/NAMESPACE/SECCOMP` decision, and the audit trail will faithfully record a false provenance for it — the log looks legitimate even though nothing verified the claim.

Suggested fix. Enforce `AuthorizedBy` against a configured allowlist of known agent identities before acting on destructive actions, or use real transport-level authorization (e.g. `SO_PEERCRED` UID checks on the UDS socket) instead of a client-supplied string.

BUG-011 — No Protected-PID Gate on the Go Control-Plane Side

Severity: High (Security)
Component: `kb-control-plane/internal/enforcement/enforce.go(Contain),calledf`

romSetContainmentandSubmitAgentDecision
Status: Open

Description. kb-core's CPM (Critical Process Module) correctly gates containment writes on the C sensor side — refusing to contain PID 1, kernel threads, or protected executables. But the Go control plane is the *other* entry point a containment request can originate from (operator RPC, agent decision), and it has **zero exemption logic of its own**.

Confirmed in source. Grepping enforce.go for any PID-1/self-PID/protected-list check returns nothing — Contain() forwards any PID it is given exactly the same way, relying entirely on kb-core's gate one hop downstream over kbct.sock.

Impact. A single bug or race in kb-core's CPM gate has no backstop on the Go side. A request to contain PID 1, systemd, or kbd's own PID is forwarded identically to a request against any other process.

Suggested fix. A minimal check in Contain() (or immediately before it) refusing/alerting on requests targeting PID 1, os.Getpid() (the control plane itself), and a small configured list of critical process names — independent of whatever kb-core does downstream.

BUG-012 — No Input Validation or Rate Limiting on Containment RPCs

Severity: Medium (Security/Robustness)
Component: kb-control-plane/internal/controlplane/grpc.go,SetContainment+SubmitAgentDecision
Status: Open

Description. Both RPCs accept any int32/uint32 Pid (including 0, a negative cast, or a PID that does not exist) and act on it without checking existence against the store first. Neither has any throttle.

Confirmed in source:

```
func (cp *ControlPlane) SetContainment(ctx context.Context,
    req *pb.ContainmentRequest) (*pb.ContainmentResponse, error) {
    cp.enforcer.Contain(req.Pid, uint32(req.Level), req.Reason)
    ...
```

No store.GetProcessState existence check precedes the Contain() call in either RPC; no rate-limiter or token bucket exists anywhere in this file.

Impact. A buggy or compromised agent client can call SubmitAgentDecision in a tight loop with no backpressure, and containment actions can be issued against PIDs the store has never even observed.

Suggested fix. Validate Pid exists via store.GetProcessState before acting; add a simple per-agent-ID token-bucket or minimum-interval check.

BUG-013 — SSH Listener Has No Auth-Attempt Throttling; Dev-Mode Fallback Silently Weakens Key Material

Severity: Medium-High (Security)
Component: `kb-control-plane/internal/ssh/server.go,internal/ssh/config.go`
Status: Open (architectural fix planned, not yet implemented)

Description. This is the daemon's only network-exposed listener (port 2222) — every other socket is UDS, local-only. Two distinct issues:

1. **No rate limiting/lockout on failed public-key attempts.** `wish.WithPublicKeyAuth` calls `ValidatePublicKey` per attempt with no counter, backoff, or fail2ban-style hook. Public-key auth makes brute-force cryptographically impractical, but there is no protection against connection-exhaustion/resource-abuse from repeated handshakes.

2. **Dev-mode fallback is a real security downgrade, not just a convenience.** When `/etc/kb` is not writable and `KB_DEV=true`, `LoadConfig()` silently switches to host keys and an `authorized_keys` file stored **relative to the process's current working directory**. If `kbd` is ever started with `KB_DEV=true` from an unexpected working directory (a misconfigured systemd unit, a CI job, a debugging session on a semi-production box), the host key silently regenerates or resolves somewhere unexpected — breaking host-key pinning for anyone who previously trusted the old fingerprint.

Impact. An attacker with network reach to port 2222 has unlimited authentication attempts against the SSH listener; a misconfigured dev-mode deployment can silently rotate host keys in a way that trains operators to ignore host-key-changed warnings (classic MITM setup).

Suggested fix. Short-term — per-IP attempt counter/backoff, and require an explicit second opt-in (e.g. `KB_SSH_ALLOW_DEV_FALLBACK=true`) beyond just `KB_DEV=true` before falling back to CWD-relative keys. Longer-term (already decided, not yet built): replace the custom Go SSH server with real OpenSSH + `ForceCommand`, inheriting ~25 years of hardening instead of re-implementing session/auth handling in a far less battle-tested library.

BUG-014 — kb-tui Launch Path Is a Hardcoded, Single-Developer Absolute Path

Severity: Medium (Deployment)
Component: `kb-control-plane/internal/ssh/session.go`
Status: Open

Description. The SSH session handler locates the `kb-tui` binary via a hardcoded fallback chain that includes an absolute path specific to one developer's machine, pointing at a Cargo **debug** build.

Confirmed in source:

```
tuiPath := os.Getenv("KB_TUI_PATH")
// falls back to:
"/home/emergence/Desktop/kernel-borderlands/kb-op/kb-tui/target/debug/kb-tui",
```

```
"/home/emergence/Desktop/kernel-borderlands/kb-op/kb-tui/kb-tui",
"kb-tui",
```

Impact. Deployed to any other host, every SSH session either silently falls through to a `$PATH` lookup (works only if someone manually installed `kb-tui` there) or fails outright at SSH-login time — there is no build- or install-time contract keeping `kb-control-plane` (Go) and `kb-tui` (Rust) in sync, since they are two separately-built subsystems glued together only by this file path.

Suggested fix. Standardize on one fixed install-time location (e.g. `/usr/local/bin/kb-tui`), matching how `kbd` itself would be installed. Keep only `KB_TUI_PATH` (override) and the fixed path (common case); drop the dev-machine absolute path and bare-`$PATH` fallbacks entirely. Fail loudly with a specific message instead of silently trying three guesses.

BUG-015 — kbctl audit verify Bypasses Its Own Deferred Cleanup on a Detected Chain Break

Severity: Low
Component: `kb-op/kbctl/cmd_audit.go,auditVerifyCmd`
Status: Open

Description. On a detected tamper (`!resp.ChainIntact`), the handler calls `os.Exit(1)` directly:

```
fmt.Printf("CHAIN BROKEN -- %d entries verified before break\n",
    resp.EntriesVerified)
if resp.Error != "" {
    fmt.Println(resp.Error)
}
os.Exit(1)
return nil
```

`os.Exit()` terminates the process immediately without running any deferred calls — so the `defer conn.Close()` and `defer cancel()` registered earlier in the same function never execute.

Impact. Minor in practice (a short-lived CLI process, the OS reclaims the socket/fd on exit regardless), but it is a real defect and notably sloppy in exactly the one code path whose entire purpose is flagging tamper events — the most security-relevant command in the CLI has the least-clean shutdown of any command in the file.

Suggested fix. `return fmt.Errorf(...)` and let `main()`'s existing `os.Exit(1)` (after printing to stderr) handle the non-zero exit, same as every other command in this CLI already does — rather than calling `os.Exit` directly from inside `RunE`.

2 Resolved Since the Gaps Were First Documented

These were open at the time `control-plane-catalog.md` was written but are confirmed fixed in current source — included to show real progress, not just outstanding issues.

RESOLVED-1 — Audit Hash-Chain Verification Was Implemented but Never Wired to Anything Callable

Was: `Logger.VerifyChain()` existed and was correct, but had zero callers outside its own test — tampering with the audit log in SQLite directly would never be detected or alerted on.

Now: Confirmed wired on both paths — `GET /api/audit/verify` (internal/controlplane/http.go) and a `VerifyAuditChain` gRPC RPC (`internal/controlplane/grpc.go`, `proto/kb.proto`), plus a startup check in `controlplane.go`.

RESOLVED-2 — GetProcessState Did Not Distinguish "Not Found" From "Zero Value"

Was: A PID never tracked by the store returned an empty-but-valid `ProcessState` struct indistinguishable from a real all-defaults process.

Now: Confirmed fixed — `GetProcessState` returns `status.Errorf(codes.NotFound, "no tracked process with pid=%d", req.Pid)` when the store lookup misses.

RESOLVED-3 — Policy Had No Hot-Reload Path on the Go Side

Was: `control-plane-catalog.md` §2.6 documented that changing `policy.yaml` required a full `kbd` restart — the Go-side `Engine` had no reload path at all.

Now: Confirmed fixed and correctly implemented — `kbctl policy reload` → `ReloadPolicy` gRPC handler → `reloadPolicyFromDisk()` builds a fresh `policy.Engine` and swaps it into `cp.policy` under a proper `sync.RWMutex`. The write side takes a full lock; the read side (in the zone-transition handler) takes a read lock — genuinely race-free, not just apparently working. Found while auditing `kbctl` for BUG-015; initially suspected a data race here, traced it fully, and it checks out.

3 Historical — Critical Bugs From CHANGELOG.md, Already Fixed and Verified Live

HIST-1 — sudo/PAM System-Wide Lockout During Live LSM Enforcement

Severity: Critical
Component: `kb-core/ebpf/kbd_sensor.bpf.c`
Status: Fixed

Root cause: `kb_lsm_file_open`'s sensitive-path block was unconditional — applied to every process, not just contained ones — inconsistent with every other LSM hook in the same file. The first time it fired correctly, it blocked `/etc/sudoers`/`/etc/shadow` reads system-wide, breaking `sudo`/`su`/`pkexec`/login PAM simultaneously, with no already-authenticated root path left inside the machine to recover from — required a full external VM restart.

Fix: Made the block containment-gated (level ≥ 2 only), matching the rest of the file.

Verification: `sudo whoami` succeeds with the sensor actively enforcing; a deliberately-contained test process still has its access correctly blocked.

HIST-2 — SetContainment Silently Never Applied Containment at All

Severity: Critical
Component: `kb-control-plane/kb-corewireprotocol`
Status: Fixed

Root cause: `MsgTypeContainmentCmd` was `3` on the Go side but the C sensor checked for `KB_WIRE_MSG_CONTAINMENT_CMD` (`5`) — every containment command from `kb-tui`/`kbctl` was silently dropped, while `kbd` logged a successful audit entry regardless (the sensor never NACKs), completely masking the failure.

Fix: Corrected the Go-side constant to `5`.

Verification: A test PID placed into Seccomp containment via `kb-tui` had its sensitive-path access correctly blocked afterward; an uncontained process reading the same file continued to succeed.

HIST-3 — Sensitive-Path LSM Block Never Actually Enforced, Despite Being Documented as Active

Severity: High
Component: `kb-core/ebpf/kbd_sensor.bpf.c`
Status: Fixed

Root cause: `bpf_d_path()` writes a NUL-terminated path but does not guarantee the buffer's tail past the terminator is zeroed; the map lookup compared the full fixed-size 64-byte key including that uninitialized tail, so even an exact, correctly-registered path never matched.

Fix: Explicitly zero-fill `path_buf[len..63]` after `bpf_d_path()` returns.